RMC — Reconfigurable DDR5 Memory Controller

MC Core Internal Block Definitions
Version 1.9.8 — Complete Updated Specification

Capstone Design Document

Contents

1 Document Scope and Parameter Convention

This document defines all internal blocks of the MC Core domain (post-CDC, MC clock domain). All bit-widths are compile-time parameters — no hardcoded values appear anywhere in this specification.

1.1 Parameter Definitions

Parameter	Meaning
GC_WIDTH	Global cycle counter width
ADDR_WIDTH	Byte address width
AXI_ID_WIDTH	AXI transaction ID width
DATA_WIDTH	Data bus width (payload)
STRB_WIDTH	Byte strobe width = DATA_WIDTH/8
CH_BITS	Channel select bits
RANK_BITS	Rank select bits
BG_BITS	Bank group bits
BANK_BITS	Bank bits
ROW_BITS	Row address bits
COL_BITS	Column address bits
N_RANKS	Number of ranks
N_BG	Number of bank groups = $2^{\texttt{BG\_BITS}}$
N_BANKS	Banks per rank = $\texttt{N\_BG} \times 2^{\texttt{BANK\_BITS}}$
N_WR_ENTRIES	Write buffer depth (2–3× N_RD_ENTRIES)
N_RD_ENTRIES	Read buffer depth
WR_BUF_DEPTH	Write data buffer depth
STATUS_WIDTH	Request status field width
TIMING_WIDTH	Timing parameter register width
PARAM_ID_WIDTH	Timing param address bits
FAW_DEPTH	Four-activate window depth (= 4, JEDEC fixed)

2 Interface Protocol Summary

Interface	Protocol	Reason
AXI4 ports	Valid-ready	Spec mandated
Async REQ FIFO write (CIF)	Credit-based push	No combinational wr_full
Async REQ FIFO read (MC)	Valid-credit receive	Registered rd_valid
Async RESP FIFO write (MC)	Credit-based push	gate_resp_fifo_avail
Async RESP FIFO read (CIF)	Valid-credit receive	Registered rd_valid
Credit return REQ (MC→CIF)	Registered 1b sync	Crosses CDC safely
Credit return RESP (CIF→MC)	Registered 1b sync	Crosses CDC safely
All intra-MC	Valid-credit	No back-pressure in critical path
DFI	DFI 5.2 protocol	Spec mandated
PHY→MC rddata	Valid-only	No back-pressure on return

3 TCAM Blocks

3.1 WR_TCAM — Write Address CAM

Used for RAW hazard detection. Full address exact match. **No valid or timestamp fields in TCAM entries.** Valid gating: `WR_TCAM match[i] AND wr_status_reg[i].valid`.

Field	Width	Description
bg	BG_BITS	Bank group (exact match)
bank	BANK_BITS	Bank (exact match)
row	ROW_BITS	Row address (exact match)
col	COL_BITS	Column address (exact match)
req_type	1b	Always WR=1
axi_id	AXI_ID_WIDTH	AXI transaction ID
entry_idx	$clog2(N_WR_ENTRIES)	Index into WR status reg
data_buf_idx	$clog2(WR_BUF_DEPTH)	Optional index into Write Data Buffer

Cell count: N_WR_ENTRIES $\times$ 32b $\times$ 12T = 24,576T (baseline)

3.2 RD_TCAM — Read Address CAM

Used by Scheduler Stage 1 as bank pre-filter. Ternary search on {BG, bank} only.

Field	Width	Description
bg	BG_BITS	Bank group (exact match)
bank	BANK_BITS	Bank (exact match)
row	ROW_BITS	Carried, not matched
col	COL_BITS	Carried, not matched
req_type	1b	Always RD=0
axi_id	AXI_ID_WIDTH	AXI transaction ID
entry_idx	$clog2(N_RD_ENTRIES)	Index into RD status reg

Cell count: N_RD_ENTRIES $\times$ 5b $\times$ 12T = 1,920T (baseline)

Multi-hit within same bank: winner = argmax(age[i]) via status reg lookup.

4 Status Register Files

4.1 WR Status Register

Owner: Write Watermark Buffer Manager (R/W). **Scheduler:** READ ONLY.

Field	Width	Description
valid	1b	Entry occupied. Also gates WR_TCAM match output.
status	STATUS_WIDTH	00=PENDING, 01=ISSUED, 10=DONE, 11=ERROR
age	GC_WIDTH	Allocation timestamp = gc at alloc time

Note: `valid` is the single source of truth for occupancy. When `valid[i]==0`, WR_TCAM match[i] is suppressed — invalid rows never evaluate, enabling dynamic power reduction at low occupancy.

4.2 RD Status Register

Owner: Read Watermark Buffer Manager (R/W). **Scheduler:** READ ONLY.

Field	Width	Description
`valid`	1b	Entry occupied. Gates RD_TCAM match output.
`status`	`STATUS_WIDTH`	00=PENDING, 01=ISSUED, 10=DONE, 11=ERROR
`age`	`GC_WIDTH`	Allocation timestamp
`merge_pending`	1b	RAW partial overlap: DRAM fetch in progress
`merge_wdb_idx`	`$clog2(WR_BUF_DEPTH)`	Valid when merge_pending=1

`age` is used for: multi-hit newest-wins arbitration, starvation check, SJF cost computation.

5 Write Data Buffer

SRAM macro, random access (not FIFO). Index-addressed by `data_buf_idx`.

Field	Width	Description
`data`	`DATA_WIDTH`	Write payload
`mask`	`STRB_WIDTH`	Byte-level write mask

Depth: `WR_BUF_DEPTH` entries. Size: `WR_BUF_DEPTH` × (`DATA_WIDTH` + `STRB_WIDTH`) bits.

6 RAW Bypass Manager

6.1 Overview

Position: between RD request allocation and Scheduler. Runs every clock cycle via multi-port WR_TCAM search.

Hit valid condition: `wr_status_reg[hit_idx].age <= rd_age` (write arrived same cycle or earlier than read).

CRITICAL: RAW hit completes READ only. Matched write entry untouched — proceeds full PENDING→ISSUED→DONE→WR_ACK lifecycle.

6.2 Stage A — WR_TCAM Search (cycle 0, combinational with RD alloc)

```
search key: {BG, bank, row, col} -- exact match
           axi_id masked (don't care)
BL16 alignment guaranteed -> no col LSB masking needed
hit vector gated by wr_status_reg[i].valid
multi-hit: winner = argmax(age[i]) via status reg
```

6.3 Stage B — Coverage Check (cycle 1)

Case	Action
Full hit (`wdb_mask == all-1`)	Forward WDB data directly to resp FIFO. Fastest path, no DRAM.
Partial overlap (gaps in coverage)	Issue RD to scheduler immediately (NO STALL). Set `merge_pending` + `wdb_entry_idx` in RD status reg. Merge unit combines WDB + DRAM at return. Cost = normal DRAM latency.
Overshoot (write covers more)	Mask to read's required bytes, forward covered portion only.
Miss or write arrived late	Pass RD to scheduler normally. Zero added latency.

6.4 Merge Unit

```
merged[byte] = wdb_data[byte] if wdb_mask[byte] == 1
             dram_data[byte] otherwise

64 x 2:1 mux array -- combinational, one cycle at dram_valid
```

6.5 2-Deep Hold-and-Forward

Two independent response sources can fire same cycle: `src0` = RAW Bypass, `src1` = Read Data Path (DRAM return).

3rd collision is **impossible by construction**: each source is rate-limited to 1 response per cycle (single req FIFO output, single `dfi_rddata_valid`). 2-deep is exact minimum and maximum required.

Field	Width	Description
`hold_slot[1:0]`	2 × resp packet	Two independent hold slots
`hold_valid[1:0]`	2b	Validity per slot

7 Per-Bank FSM Table

Instances: `N_RANKS` × 16 rows.

7.1 Fields (Locked)

Field	Width	Description
`state`	`BANK_STATE_WIDTH`	Bank FSM state encoding
`row_open`	`ROW_BITS`	Currently open row address
`next_cas`	`GC_WIDTH`	Deadline timestamp: ACT→CAS
`next_pre`	`GC_WIDTH`	Deadline timestamp: CAS→PRE
`next_act`	`GC_WIDTH`	Deadline timestamp: PRE→ACT
`next_ref`	`GC_WIDTH`	Deadline timestamp: REFsb recovery
`can_cas`	1b	Registered flag: `(gc - next_cas)[GC_WIDTH-1] == 0`

can_pre	1b	Registered flag: (gc - next_pre)[GC_WIDTH-1] == 0
can_act	1b	Registered flag: (gc - next_act)[GC_WIDTH-1] == 0
can_ref	1b	Registered flag: (gc - next_ref)[GC_WIDTH-1] == 0
ref_pending	1b	Set by Maintenance Engine (no TCAM equivalent for REF)

can_* update rule (every cycle, background, not in critical path):

```
can_act <= (gc - next_act)[GC_WIDTH-1] == 0
can_pre <= (gc - next_pre)[GC_WIDTH-1] == 0
can_cas <= (gc - next_cas)[GC_WIDTH-1] == 0
can_ref <= (gc - next_ref)[GC_WIDTH-1] == 0
```

Stage 2 reads can_* flags only. No subtractor in the scheduling critical path.

7.2 Removed Fields

The following fields were removed in v1.8.2 and are replaced by TCAM output:

Old Field	Replacement
open_pending	tcam_out[bank].hit
pre_pending	Stage 2 computes: tcam_out[b].row != row_open[b]
act_pending	Stage 2 checks: state[b] == IDLE
wr_pending	tcam_out[b].req_type == WR
rd_pending	tcam_out[b].req_type == RD

7.3 States

Encoding	State	Description
000	IDLE	Bank precharged, no open row
001	ACTIVATING	ACT issued, waiting tRCD
010	ACTIVE	Row open, CAS eligible
011	PRECHARGING	PRE issued, waiting tRP
100	REFRESHING_SB	REFsb issued, waiting tRFCsb
101	RFM_ACTIVE	RFM issued, waiting tRFM
110	POWER_DOWN	Rank-level PD asserted
111	SELF_REFRESH	Rank-level SR asserted

7.4 Key Transitions

From	To	Trigger	Writeback
IDLE	ACTIVATING	ACT issued	next_cas=gc+tRCD, next_pre=gc+tRAS
ACTIVATING	ACTIVE	can_cas==1	row_open=req.row
ACTIVE	PRECHARGING	Row miss, no hits	next_act=gc+tRP

ACTIVE	ACTIVE	CAS row hit	next_pre updated
PRECHARGING	IDLE	can_act==1	—
IDLE	REFRESHING_SB	ref_pending AND can_ref	next_ref=gc+tRFCsb
REFRESHING_SB	IDLE	can_ref==1	ref_pending=0

8 Per-Rank FSM Table

Instances: N_RANKS rows.

8.1 Fields

Field	Width	Description
state	RANK_STATE_WIDTH	Rank FSM state
next_rfc	GC_WIDTH	REFab recovery deadline
next_zq	GC_WIDTH	ZQcal recovery deadline
next_xp	GC_WIDTH	PDX recovery deadline
next_xs	GC_WIDTH	SRX recovery deadline
next_trefi	GC_WIDTH	Next tREFI expiry
next_zqcs	GC_WIDTH	Next ZQcal interval
can_rfc	1b	Registered: REFab recovery complete
can_zq	1b	Registered: ZQcal recovery complete
can_xp	1b	Registered: PDX recovery complete
can_xs	1b	Registered: SRX+DLL recovery complete
gate_rfc	1b	Blocks ALL per-bank cmds this rank
gate_zq	1b	Blocks ALL per-bank cmds this rank
ref_credits	CREDIT_WIDTH	Leaky bucket (0..15)
raa[16]	RAA_WIDTH×16	Per-bank RAA counters
last_TUF	1b	Last read TUF bit from MR4
next_poll_gc	GC_WIDTH	Next MRR poll deadline
mrr_data	8b	Last MR4 response
last_refsb_gc[32]	GC_WIDTH×32	Timestamp of last REFsb per bank

8.2 States

Encoding	State	Description
000	NORMAL	Rank operational
001	REFab_ACTIVE	REFab issued, all banks blocked
010	ZQCAL_ACTIVE	ZQcal in progress
011	POWER_DOWN	Rank in PD
100	SELF_REFRESH	Rank in SR

9 Global Timing Table

Instances: 1.

Field	Width	Description

global_state	GL_STATE_W	INIT/NORMAL/ SOFT_RESET/ REF_STALL/ ZQ_STALL
next_act_any	GC_WIDTH	tRRD_S from last ACT anywhere
next_cas_any	GC_WIDTH	tCCD_S from last CAS anywhere
next_rd_wr	GC_WIDTH	tWTR_S from last WR data-end
next_wr_rd	GC_WIDTH	tRTW from last RD
faw_window[FAW_DEPTH]	GC_WIDTH×N	Ring buffer of recent ACT timestamps
next_act_bg[N_BG]	GC_WIDTH×N_BG	tRRD_L per BG
next_cas_bg[N_BG]	GC_WIDTH×N_BG	tCCD_L/tCCD_L_WR per BG
next_wtr_bg[N_BG]	GC_WIDTH×N_BG	tWTR_L per BG
last_act_bg[N_BG]	GC_WIDTH×N_BG	gc at last ACT per BG
next_act_rank[N_RANKS]	GC_WIDTH×N_R	tRRD within same rank
next_cas_rank[N_RANKS]	GC_WIDTH×N_R	tCCD within same rank
can_act_any	1b	Registered flag
can_cas_any	1b	Registered flag
can_rd_wr	1b	Registered flag
can_wr_rd	1b	Registered flag
can_faw	1b	Registered flag
can_act_bg[N_BG]	N_BG bits	Registered flags
can_cas_bg[N_BG]	N_BG bits	Registered flags
can_wtr_bg[N_BG]	N_BG bits	Registered flags
can_act_rank[N_RANKS]	N_RANKS bits	Registered flags
can_cas_rank[N_RANKS]	N_RANKS bits	Registered flags

10 Bank Activity Counter Table

Instances: N_RANKS × 16 rows.

Field	Width	Description
count	$clog2(BUF_DEPTH+1)	Outstanding requests to this bank
dirty	1b	1 = pending WR entry exists

Updates: count++ on alloc, count-- on retire. dirty=1 on WR alloc, dirty=0 when last WR retires.

Usage:

- Maintenance Engine: REFsb targets argmin(count) across rank
- Power Management: PD entry when count==0 all banks
- Scheduler Stage 3: prefer high-count banks (keep row open)
- Bank Partition Controller: WR override when count pressure high

11 timing_reg_file

Property	Value
Addressing	PARAM_ID_WIDTH-bit enum
Entry width	TIMING_WIDTH bits
Read	Combinational, multi-port
Write	CSR only (init time, slow path)

Command → timing update vector (parallel adders):

Command	Parameters Updated
ACT	tRCD, tRAS, tRRD_L, tRRD_S, tFAW
CAS_RD	tCCD_L, tCCD_S, tRTP, tWTR_L, tWTR_S, tRTW
CAS_WR	tCCD_L_WR, tCCD_L_WR2, tWR, tWTR_L, tWTR_S
PRE	tRP
REFab	tRFC1
REFsb	tRFCsb

12 Scheduler Pipeline

12.1 Stage Overview

```
Request Buffers [N_WR/RD_ENTRIES]
       |
  [ Stage 0 ] Maintenance Override (REF > ZQ > RFM bypass)
       |
  [ Stage 1 ] TCAM Search -> hit_bitmap + metadata
       |
  [ Stage 2 ] can_* Gate Check + Cost Classification
       |          + Partition Mask Application
       |          + Speculative ACT detection
       |
  [ Stage 3 ] SJF Winner Selection
       |
  [ Stage 4 ] Command Emission + All Table Writebacks
```

12.2 Stage 2 — Legal Check Matrix

All checks use registered can_* flags. No subtractors in critical path.

Gate	Source	Constraint
can_cas[b]	Per-bank registered	tRCD elapsed
can_pre[b]	Per-bank registered	tRAS/tRTP/tWR elapsed
can_act[b]	Per-bank registered	tRP elapsed
can_ref[b]	Per-bank registered	tRFCsb elapsed
can_act_bg[bg]	Global registered	tRRD_L
can_act_any	Global registered	tRRD_S
can_faw	Global registered	$\leq$4 ACTs in tFAW
can_cas_bg[bg]	Global registered	tCCD_L/tCCD_L_WR

can_cas_any	Global registered	tCCD_S
can_rd_wr	Global registered	tWTR_S
can_wr_rd	Global registered	tRTW
can_act_rank[r]	Global registered	tRRD within rank
can_cas_rank[r]	Global registered	tCCD within rank
gate_rfc[r]	Per-rank	REFab blocking
gate_zq[r]	Per-rank	ZQcal blocking
gate_resp_fifo_avail	Resp FIFO	Free slot before RD issues

12.3 Stage 2 — Cost Classification (SJF)

Bank State	Remaining Cost
ACTIVE, row hit	0 (immediate CAS)
ACTIVATING	next_cas[b] - gc
IDLE	tRCD
ACTIVE, row miss	(next_pre[b] - gc) + tRP + tRCD
PRECHARGING	(next_act[b] - gc) + tRCD

12.4 Stage 3 — Priority Order

1. Stage 0 override (REF/ZQ/RFM)
2. STARVED_MISS: age[i] $\geq$ RD_STARVATION_THR + entry_idx[i]
3. HIT_SET (cost=0): prefer bg != last_act_bg[rank]
4. MISS_SET: lowest remaining_cost

Starvation thresholds:

- RD_STARVATION_THR = 12,480 cycles (CSR, default 9×tREFI)
- WR_STARVATION_THR = 37,440 cycles (CSR, 3× RD threshold)

Stagger: starved[i] = age[i] $\geq$ THR + entry_idx[i] ensures at most one fires per cycle.

12.5 Bank Partition RD/WR Scheme

Field	Width	Description
partition_reg	1b	Current partition config (A or B)
window_ctr	GC_WIDTH	Countdown to rotation
rd_partition_mask	N_BANKS	Banks serving RD this window
wr_partition_mask	N_BANKS	Banks serving WR this window
WINDOW_SIZE	GC_WIDTH	CSR, default 2×tREFI

No tRTW or tWTR within window. Penalty only at rotation boundary (amortized over WINDOW_SIZE).

12.6 Speculative Prefetch ACT

Fires on NOP cycles only (winner_valid==0):

```
for each ACTIVE bank[r][bg][b]:
  col_remaining = COL_MAX - cur_col
  if col_remaining <= BL_BYTES:    -- page boundary imminent
    pred = {rank=r, bg=bg, bank=b+1, row=row_open[r][bg][b]}
    if RD_TCAM hit for {bg, b+1}
    AND tcam_out[b+1].row == pred.row -- same row confirmed
    AND bank_act_count[r][bg][b+1] > 0 -- real work exists
    AND state[r][bg][b+1] == IDLE
    AND can_act[r][bg][b+1]
    AND can_faw:
      -> emit speculative ACT to pred bank
```

12.7 NOP Cycle Priority (Final)

1. Opportunity REFsb (correctness first)
2. Speculative prefetch ACT
3. WR partition drain (row hit in WR partition)
4. True NOP

12.8 Scheduler Invariants

1. Row hit always beats row miss (unless starved)
2. At most one ACT per cycle (faw_window enforces 4-ACT limit)
3. At most one CAS per cycle
4. REF/ZQ/RFM override Stages 1–3 via Stage 0
5. Starved miss: at most one fires per cycle (stagger guarantee)
6. RD never issues without `gate_resp_fifo_avail`
7. WR never issues without valid `data_buf_idx`
8. No cmd issues while `gate_rfc[rank]` or `gate_zq[rank]` asserted
9. Consecutive ACTs prefer different BG (tRRD_S ¡ tRRD_L)
10. CAS issued only when `can_cas[b]==1`
11. REFab: rank already idle, no bank-to-bank delay needed
12. REFsb: PRE required if target bank ACTIVE
13. SJF: lowest `remaining_cost` wins in MISS_SET
14. Stage 2 reads only registered `can_*` flags, no subtractor in critical path
15. WR_TCAM RAW hit valid only if `wr_age <= rd_age`
16. TCAM match suppressed when `status_reg[i].valid==0`
17. Speculative ACT fires only on true NOP cycles
18. Speculative ACT requires RD_TCAM confirmation
19. Speculative ACT requires `bank_act_count[next_bank] > 0`
20. Speculative ACT subject to same `can_faw` gate as real ACT

21. Opportunity REFsb fires before speculative ACT on NOP cycles
22. `last_refsb_gc` updated on every REFsb regardless of trigger
23. Watchdog overrides argmin if any bank overdue by `tREFI` × 32
24. RD and WR never share same bank in same partition window
25. Partition rotation penalty (tRTW or tWTR_L) occurs once per window

13 Maintenance Engine

13.1 Sub-FSM List (6 total)

#	FSM	Description
1	Init FSM	16-state DDR5 power-up sequence. Owns DFI exclusively until init_done.
2	Refresh FSM	6 states. Leaky-bucket, REFab/REFsb/FGR. Opportunity refresh.
3	ZQcal FSM	7 states per rank. MPC ZQCAL Start→Latch.
4	RFM FSM	6 states. Per-bank RAA counters, RAAIMT threshold.
5	Power Mgmt FSM	10 states. PD and SR branches.
6	MR_Poll FSM	6 states. Periodic MR4 TUF read. Temperature-aware tREFI.

13.2 DFI Output Mux (inside ME)

```
init_done = 0 -> Init FSM drives all DFI outputs
init_done = 1 -> Scheduler Stage 4 drives all DFI outputs

init_done: one-way latch, never de-asserted after boot
MRR during operation: MR_Poll FSM -> Stage 0 bypass -> Stage 4 -> DFI
no third mux input needed
```

13.3 Internal Priority (ME arbitration)

`ref_urgent` > `ref_due` > `rfm_req` > `zq_due`

One `me_cmd_valid` asserted per cycle maximum.

13.4 Refresh FSM

States: IDLE → REF_DUE → WAIT_BANKS_IDLE → ISSUE_REF → WAIT_tRFC → DONE → IDLE

Leaky-bucket: credits++ per tREFI, credits– per REF issued. `ref_urgent` when credits ≥ 8.

REFsb targeting:

- Normal: `argmin(bank_act_count[rank][*])` — least-loaded bank first

- Watchdog: if `gc - last_refsb_gc[b] > tREFI×32` — force that bank

Opportunity refresh: fires on NOP cycles when `bank_act_count==0` and bank most overdue.

13.5 MR_Poll FSM (Sub-FSM 6)

States: IDLE → WAIT_INTERVAL → REQUEST_MRR → WAIT_RDDATA → PARSE_TUF → UPDATE_TREFI → IDLE

TUF handling:

- TUF=0: `tREFI_adjusted = tREFI`
- TUF=1 (>85°C): `tREFI_adjusted = tREFI / 2`

MRR response received via sideband path from Read Data Path (not through resp FIFO). Default polling interval: `MRR_POLL_INTERVAL` CSR = 32×tREFI.

14 Address Map Unit (AMU)

Replaces the Address Translator in the CIF domain. Configurable at setup time via CSR. Locked before `init_done`.

14.1 Pipeline

```
Stage 1 (combinational): XOR hash (per-field opt-in)
  hashed_addr[field] = raw_addr XOR (raw_addr >> xor_shift[field])

Stage 2 (combinational): field extract from hashed_addr
  supports split fields (non-contiguous bit ranges)
```

14.2 Field Descriptor (per field, 6 fields)

Field	Width	Description
`src_msb_a`	5b	Upper segment MSB in address
`src_lsb_a`	5b	Upper segment LSB
`src_msb_b`	5b	Lower segment MSB (split only)
`src_lsb_b`	5b	Lower segment LSB (split only)
`split_en`	1b	1=use both segments, 0=single range
`hash_en`	1b	Per-field XOR hash enable
`xor_shift`	5b	Per-field shift amount

14.3 Field Assignment Policy (locked)

Field	hash_en	Rationale
`rank`	1 (MSB, highest bits)	Spread across ranks, rank switch rare
`ch`	1	Spread across channels, same locality
`bg`	0	Preserve locality, row hit rate
`bank`	0	Preserve locality
`row`	0	Preserve locality
`col`	0	Preserve locality, split_en for ch interleave

Channel interleave granularity (OQ-19b): TBD, requires traffic trace analysis. CSR configurable.

15 Config Registers (Key)

Register	Type	Default	Description
WR_HIGH_WM	CSR	16	WR drain entry watermark
WR_LOW_WM	CSR	4	WR drain exit watermark
AGE_THR1	CSR	64	Intra-class HOL bypass
AGE_THR2	CSR	256	Cross-class mode flip
RD_STARVATION_THR	CSR	12,480	RD miss forced service
WR_STARVATION_THR	CSR	37,440	WR miss forced service (3× RD)
WINDOW_SIZE	CSR	2×tREFI	RD/WR partition window
PAGE_POLICY	CSR	00	00=Open, 01=Closed, 10=Adaptive
REF_MODE	CSR	00	00=REFab, 01=REFsb, 10=FGR-2x, 11=FGR-4x
MRR_POLL_INTERVAL	CSR	32×tREFI	MR4 TUF polling interval
PD_IDLE_THRESHOLD	CSR	64	Cycles idle before PD entry
FIFO_DEPTH	param	16	Async FIFO depth (initial credits)
INIT_KICK	CSR	0	Trigger Init FSM
SOFT_RESET	CSR	0	Sync soft-reset all FSMs

16 FSM Count Summary

FSM	States	Instances	Owner
Init FSM	16	1	ME sub-FSM 1
Refresh FSM	6	1	ME sub-FSM 2
ZQcal FSM	7	N_RANKS	ME sub-FSM 3
RFM FSM	6	1	ME sub-FSM 4
Power Mgmt FSM	10	1	ME sub-FSM 5
MR_Poll FSM	6	1	ME sub-FSM 6
Per-Bank FSM	8	16×N_RANKS	Scheduler
Per-Rank FSM	5	N_RANKS	ME
Global FSM	5	1	Global
Write CRC FSM	4	1	Write Data Path
ECS FSM	4	1	Read Data Path
Bank Partition FSM	2	1	Scheduler

17 Block Ownership Summary

Block	Owner (R/W)	Scheduler Access
WR Status Reg	Write WM Manager	READ ONLY
RD Status Reg	Read WM Manager	READ ONLY
WR_TCAM	Write WM Manager	READ ONLY (search)
RD_TCAM	Read WM Manager	READ ONLY (search)
Write Data Buffer	Write WM Manager	READ ONLY (emit)
Per-Bank FSM Table	Scheduler S4 + ME	WRITE S4, READ S2
Per-Rank FSM Table	ME	READ S0
Global Timing Table	Scheduler S4	READ S2
Bank Activity Counter	WM Managers	READ ME + Power Mgmt
timing_reg_file	CSR (init only)	READ ONLY
Global Cycle Counter	Free-running	READ all blocks
Bank Partition Controller	Scheduler	READ S2